Doc. no. P0003R3
Date: 2016-06-23
Project: Programming Language C++
Audience: Core Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Removing Deprecated Exception Specifications from C++17

Table of Contents

Revision History

A non-exhaustive list of changes through the revisions of this paper.

Revision 0

Original version of the paper for the 2015 pre-Kona mailing.

Revision 1

Revised the wording to modify the proposed working draft after the Kona 2015 meeting, N4567. This accommodates renumbering a few clauses, notably Annex D where several other features have already been removed, and allowing for exception specifications in the type system, and the rewording of inheriting constructors.

Revision 2

Changes for Jacksonville wording review:

- Fixed UTF8 marker for file-encoding
- Fix numerous typos
- Adopt simplified wording, rewriting the whole of clause **15.4 [except.spec]**
- Define semantics consistently in the terms of *potentially-throwing* or *non-throwing* operations, rather than whether *exceptions are allowed*
- Precise in usage of *exception-specification* vs. (plain) exception specification
- Provide list of zombie names
- (post meeting) Rebase wording off N4582, the post-meeting working paper
- (post meeting) Defer to issue 1343 to introduce definition for *immediate subexpression*
- (post meeting) Address inconsistency in the older compatibility annex

Revision 3

Changes for Oulu wording review:

- Prefer "function declarator" to "function declaration's declarator"
- Exception specifications apply to a function type, not to a function
- Remove ~~deleted~~ text that referred to previous wording, rather than current working draft
- Simplify wording related to initializing base classes, and inheriting constructors
- Reordered and merged several pragraphs in new 15.4 wording
- Fix use of array-new in the example

Introduction

Dynamic exception specifications were deprecated in C++11. This paper formally proposes removing the feature from C++17, while retaining the (still) deprecated `throw()` specification strictly as an alias for `noexcept(true)`.

A Brief History of Exception Specifications

Exception specifications were added as part of the original design of the exception language feature. However, the experience of using them was less than desired, with the general consensus by the time the 1998 standard was published being, generally, to not use them. The standard library made the explicit choice to not use exception specifications apart from a handful of places where guaranteeing an empty (no-throwing) specification seemed useful. N741 gives a brief summary of the LWG thoughts at the time.

By the time C++11 was published, the feeling against exception specifications had grown, the limited real-world use generally reported negative user experience, and the language feature, renamed as *dynamic* exception specifications, was deprecated, (N3051). A new language feature, `noexcept`, was introduced to describe the important case of knowing when a function could guarantee to not throw any exceptions.

Looking ahead to C++17, there is a desire to incorporate exception specifications into the type system, P0012R1. This solves a number of awkward corners that arise from exception specifications not being part of the type of a function, but still does nothing for the case of deprecated dynamic exception specifications, so the actual language does not get simpler, and we must still document and teach the awkward corners that arise from dynamic exception specifications outside the type system.

Proposed Changes

The recommendation of this paper is to remove dynamic exception specifications from the language. However, the syntax of the `throw()` specification should be retained, but no longer as a dynamic exception specification. Its meaning should become strictly

an alias for `noexcept(true)`, and its usage should remain deprecated.

To minimize the impact on the current standard wording, the grammar term *exception-specification* is retained, although it has only one production and could be replaced entirely with *noexcept-specification*. Alternatively, the grammar term *noexcept-specification* could be retired.

All wording dealing with *compatible* exception specifications, that used to be described in terms of sets of permitted exceptions, has been redrafted in terms of *potentially-throwing* and *non-throwing* operations and exception specifications. Eliminating the set-algebra significantly simplifies the wording.

Motivation for Change

Dynamic exception specifications are a failed experiment, but this is not immediately clear to novices, especially where the "novice" is an experienced developer coming from other languages such as Java, where exception specifications may be more widely used. Their continuing presence, especially in an important part of the language model (exceptions are key to understanding constructors, destructors, and RAII) is an impediment that must be explained. It remains embarrassing to explain to new developers that there are features of the language that conforming compilers are required to support, yet should never be used.

Exception specifications in general occupy an awkward corner of the grammar, where they do not affect the type system, yet critically affect how a virtual function can be overridden, or which functions can bind to certain function pointer variables. As noted above, P0012R1 goes a long way to resolving that problem for `noexcept` exception specifications, which makes the hole left for dynamic exception specifications even more awkward and unusual for the next generation of C++ developers.

C++17 is on schedule to be a release with several breaking changes for old code, with the standard library removing `auto_ptr`, the classic C++98 function binders, and more. Similarly, it is expected that the core language will lose trigraphs, the `register` keyword, and the increment operator for `bool`. This would be a good time to make a clean break with long discouraged (and actively deprecated) parts of the language.

The proposed change would resolve core issue 596 as no longer relevant (NAD), and should simplify core issue 1351, although that is marked as a Defect Report that was not yet applied to the working paper the proposed wording below was drafted from.

Compatibility Concerns

There is certainly some body of existing code that has not heeded the existing best practice of discouraging dynamic exception specifications, and has not yet accounted for the feature being deprecated in C++11. It is not clear how much of such code would be expected to port unmodified into a C++17 (or beyond) world, and the change is relatively simple - just strike the (non-empty) dynamic exception specification to retain equivalent meaning. The key difference is that the `unexpected` handler will not now be called to translate unexpected exceptions. In rare cases, this would allow a new exception type to propagate, rather than calling `terminate` to abort. If enforcing that semantic is seen as important in production systems, there is a more intrusive workaround available:

```
void legacy() throw(something)
try
{
    // function body as before
}
catch(const something&) {
   throw;
}
catch(...) {
   terminate();
}
```

It is thought that the empty dynamic exception specification, `throw()`, was much more widely used in practice, often in the mistaken impression that compilers would use this information to optimize code generation where no exception could propagate, where in fact this is strictly a pessimization that forces stack unwinding to the function exit, and then calling the `unexpected` callback in a manner that is guaranteed to fail before the subsequent call to `terminate`. This paper proposes treating such (still deprecated) exception specifications as synonyms for `noexcept(true)`, yielding the performance benefit many were originally expecting.

It should also be noted that at least one widely distributed compiler has still not implemented this feature in 2015, and at least one vendor has expressed a desire to never implement the deprecated feature (while that vendor *has* implemented the `noexcept` form of exception specification). Code on that platform would not be adversely impacted by the proposed removal, and portable code must always have allowed for the idiosyncrasies of this platform.

One remaining task is to survey popular open source libraries and see what level of usage, if any, remains in large, easily accessible codebases.

Review

Evolution Working Group Review

The first version of this paper was presented to the Evolution Working Group at the Kona 2015 meeting, and was recommended to advance to Core 'as written'. Specifically, that means retaining the empty exception specification as an alternate spelling of `noexcept(true)`:

SF	WF	N	WA	SA
12	13	6	0	0

Design Choices

By choosing to keep the `throw()` specification, we look to retain source compatibility with a large body of existing code that has used this specification to indicate code that should not throw. However, we also change the meaning of that specification to be exactly the same as a plain `noexcept` specification, which means that an implementation is no longer permitted to call `unexpected` when a violation is detected (which was required in all previous standards) and the previously mandated stack unwinding might no longer occur. This means that a conforming C++17 implementation is not permitted to simply retain the C++14 semantics while issuing an "extension" diagnostic.

Drafting Choices

As the author is not a regular drafter of Core wording, a quick rationale for some of the most obvious wording decisions seems helpful, especially when drawing attention to changes that were deliberately *not* made.

Where there is some question over these decisions, it would probably be more efficient to have direct feedback here, than sowing repeating detailed comments throughout the wording - at least for the first round of review.

Retain the grammar production *exception-specification*

To minimize the impact on the existing grammar and wording, the term *exception-specification* is retained at exactly the same place in the grammar. However, it loses its ability to describe anything other than a *noexcept-specification*. The empty throw specification becomes a (deprecated) synonym for `noexcept(true)`, and the *noexcept-specification* production is retired.

An alternative formulation might have been to retire the original *exception-specification* itself, forcing us to find and address each existing use in the text - either to simply replace it with *noexcept-specification*, or rewrite that part of the document to no longer refer to exception specifications at all.

The author has gone with the first option as it will be a less disruptive set of changes, and the Core working group has the expertise to spot and remove any remaining redundancies as a later clean-up (if desired).

Simplify the specification from algebra on sets of types

The current text regarding exception specifications is written in terms of sets of types that might pass through the specification, including the fictitious notion of an "any" type. With the removal of dynamic exception specifications we can distill that down to a single bit. Functions (and expressions) either allow exceptions to propagate, or they do not.

With the removal of dynamic exception specifications, why not remove the text for unwinding when an specification is violated?

The simple answer is that would be a technical change that needs to be separately blessed by EWG. The wording demanding unwinding at a violated exception specification can be cleanly removed with the whole of sub-clause 15.5.2. However, the implementation-defined choice on whether to unwind through an *exception-specification* should remain, to support the variety of exception unwinders deployed in production compilers today.

Exception specification vs. *exception-specification*

During the wording review with the Core Working Group, it was confimed that we wanted to be much more precise in the usage of the similar looking terms, "exception specification" (in plain English), refers to the notion of an exception specification, which may be implicit, while the italic term *exception-specification* denotes exactly that grammar production and its usage as such in

source files.

Considerations for Drafting Review

Does the exception specification predicate tie into the grammar?

There is some concern that the new 15.4p1 does not quite tie in the new definition for an exception specification predicate, to the grammar term that is intended to produce the value for that predicate.

Did not touch wording that refers to types in exception specifications

There are a few clauses that talk about types when used in exception specifications, such as describing the name look-up rules, or requirements for type-completeness. With the removal of *dynamic-exception-specification*s, the first attempt at drafting updated these clauses too. However, in most or all cases, the existing text remains valid, but more specific to exception specifications containing boolean constant expressions. The existing specification and notes in these paragraphs now refer only to the contents of such expressions.

For reference (and review by the diligent) the list of clauses affected by this decision to make no changes is:

- 3.3.7 p1 bullet 1 [basic.scope.class]
- 3.4.1 p7,8, footnote 31 [basic.lookup.unqual]
- 9.2 p2 [class.mem]
- 14.5 p2 [temp.decls]
- 14.6 p11 [temp.res]
- 14.6.4.1 p3 [temp.point]
- 14.7.1 p1,12 [temp.inst]
- 14.7.2 p12 [temp.explicit]
- 14.8.2 p7 [temp.deduct]

Note that all cases above use the grammar term *exception-specification* rather than the plain english "exception specification". In the non-expert view of the author drafting the paper, that seems to be correct, but would definitely be worth double-checking by a Core-drafting expert.

Clean up definition of "immediate subexpression"

The notion of an "immediate subexpression" will be more accurately defined as part of the resolution for Core Issue 1343. If that issue is resolved at the same meeting that this paper is adopted, strike the two new paragraphs 15.4p7,8. The drafting below shows those paragraphs in a stike- through font, so that the wording remains available should issue 1343 not be ready to proceed at the same meeting.

Clean up C++03 Compatibility Annex

Several paragraphs in Annex C.2 refer to dynamic exception specifications in ways that made sense for C++14, but are either no longer relevant or no longer legal in C++17.

Interaction with issues

- Core issue 596: (Open) NAD as the subclause is eliminated
- Core issue 2047: (WP) This paper totally rewrites the clause, eliminating the underlying issue
- Core issue 2183: (Open) Should be resolved in this paper's re-wording
-
- Core issue 1912: (Extension) No direct impact
- Core issue 1934: (Extension) Redraft issue as underlying wording changes significantly

Proposed Wording

First, adopt the definition of *immediate subexpression* in clause **1.9 [intro.execution]** from Core Issues 1343.

Replace the whole of clause 15.4 with:

> **15.4 Exception specifications [except.spec]**
>
> 1. The predicate indicating whether a function cannot exit via an exception is called the *exception specification* of

the function. If the predicate is false, the function has a *potentially-throwing exception specification*, otherwise it has a *non-throwing exception specification*. The exception specification is either defined implicitly, or defined explicitly by using an *exception-specification* as a suffix of a function declarator (8.3.5).

```
exception-specification:
  noexcept ( constant-expression )
  noexcept
  throw ( )
```

2. In an *exception-specification*, the *constant-expression*, if supplied, shall be a contextually converted constant expression of type `bool` (5.20 [expr.const]); that constant expression is the exception specification of the function type in which the *exception-specification* appears. A `(` token that follows `noexcept` is part of the *exception-specification* and does not commence an initializer (8.5). The *exception-specification* `noexcept` without a *constant-expression* is equivalent to the *exception-specification* `noexcept(true)`. The *exception-specification* `throw()` is deprecated (D.2), and equivalent to the *exception-specification* `noexcept(true)`.
3. If a declaration of a function has an implicit exception specification, other declarations of the function shall not specify an *exception-specification*. In an explicit instantiation (14.7.2) an *exception-specification* may be specified, but is not required. If an *exception-specification* is specified in an explicit instantiation directive, it shall be a potentially-throwing exception specification only if the exception specifications of all other declarations of that function are potentially-throwing. A diagnostic is required only if the exception specifications are not the same within a single translation unit.
4. If a virtual function has a non-throwing exception specification, all declarations, including the definition, of any function that overrides that virtual function in any derived class shall have a non-throwing exception specification, unless the overriding function is defined as deleted. [*Example:*

```
struct B {
  virtual void f() noexcept;
  virtual void g();
  virtual void h() noexcept;
};

struct D: B {
  void f();               // ill-formed
  void g() noexcept;      // OK
  void h() = delete;      // OK
};
```

The declaration of `D::f` is ill-formed because it has a potentially-throwing exception specification, whereas `B::f` has a non-throwing exception specification. — *end example*]

5. Whenever an exception is thrown and the search for a handler (15.3) encounters the outermost block of a function with a non-throwing exception specification, the function `std::terminate()` is called (15.5.1).
6. An implementation shall not reject an expression merely because, when executed, it throws or might throw an exception from a function with a non-throwing exception specification. [*Example:*

```
extern void f();  // potentially-throwing
void g() noexcept {
  f();      // valid, even if f throws
  throw 42; // valid, effectively a call to std::terminate
}
```

the call to `f` is well-formed even though, when called, `f` might throw an exception that `g` does not allow. — *end example*]

7. An expression `e` is *potentially-throwing* if
   1. `e` is function call whose *postfix-expression* (5.2.2) has a function type, or a pointer-to-function type, with a potentially-throwing exception specification, or
   2. `e` implicitly invokes a function (such as an overloaded operator, an allocation function in a *new-expression*, a constructor for a function argument, or a destructor if `e` is a full-expression (1.9)) that is potentially-throwing, or
   3. `e` is a *throw-expression* (5.17), or
   4. `e` is a `dynamic_cast` expression that casts to a reference type and requires a run-time check (5.2.7), or
   5. `e` is a `typeid` expression applied to a glvalue expression whose type is a polymorphic class type (5.2.8), or
   6. any of the immediate subexpressions (1.9) of `e` is potentially-throwing.
8. An implicitly-declared constructor for a class `X` has a potentially-throwing exception specification if and only if any of the following constructs is potentially-throwing:
   - a constructor selected by overload resolution in the implicit definition of the constructor for class `X` to initialize a potentially constructed subobject, or

- a subexpression of such an initialization, such as a default argument expression, or,
- for a default constructor, a default member initializer.

9. [*Note:* Even though destructors for fully-constructed subobjects are invoked when an exception is thrown during the execution of a constructor (15.2), their exception specifications do not contribute to the exception specification of the constructor, because an exception thrown from such a destructor would call `std::terminate` rather than escape the constructor (15.1, 15.5.1). — *end note*]
10. The exception specification for an implicitly-declared destructor is potentially-throwing if and only if any of the destructors for any of its potentially constructed subobjects is potentially throwing.
11. The exception specification for an implicitly-declared assignment operator `f` is potentially-throwing if and only if the invocation of any assignment operator in the implicit definition of `f` is potentially-throwing.
12. A deallocation function (3.7.4.2) with no explicit *exception-specification* has a non-throwing exception specification.
13. A function with an implied non-throwing exception specification, where the function's type is declared to be `T`, is instead considered to be of type "`noexcept T`".
14. [*Example:*

```
struct A {
  A(int = (A(5), 0)) noexcept;
  A(const A&) noexcept;
  A(A&&) noexcept;
  ~A();
};
struct B {
  B() throw();
  B(const B&) = default;  // implicit exception specification is noexcept(true)
  B(B&&, int = (throw Y(), 0)) noexcept;
  ~B() noexcept(false);
};
int n = 7;
struct D : public A, public B {
  int * p = new int[n];
  // D::D() potentially-throwing, as the new operator may throw bad_alloc or bad_array_new_length
  // D::D(const D&) non-throwing
  // D::D(D&&) potentially-throwing, as the default argument for B's constructor may throw
  // D::~D() potentially-throwing
};
```

    Furthermore, if `A::~A()` were virtual, the program would be ill-formed since a function that overrides a virtual function from a base class shall not have a potentially-throwing exception specification if the base class function has a non-throwing exception specification. — *end example*]

15. An exception specification is considered to be *needed* when:
    1. — in an expression, the function is the unique lookup result or the selected member of a set of overloaded functions (3.4, 13.3, 13.4);
    2. — the function is odr-used (3.2) or, if it appears in an unevaluated operand, would be odr-used if the expression were potentially-evaluated;
    3. — the exception specification is compared to that of another declaration (e.g., an explicit specialization or an overriding virtual function);
    4. — the function is defined; or
    5. — the exception specification is needed for a defaulted special member function that calls the function. [*Note:* A defaulted declaration does not require the exception specification of a base member function to be evaluated until the implicit exception specification of the derived function is needed, but an explicit *exception-specification* needs the implicit exception specification to compare against. — *end note*]

    The exception specification of a defaulted special member function is evaluated as described above only when needed; similarly, the *exception-specification* of a specialization of a function template or member function of a class template is instantiated only when needed.

And make the following additional changes to the working draft:

4.12 Function pointer conversions [conv.fctptr]

1 A prvalue of type "pointer to `noexcept` function" can be converted to a prvalue of type "pointer to function". The result is a pointer to the function. A prvalue of type "pointer to member of type `noexcept` function" can be converted to a prvalue of type "pointer to member of type function". The result points to the member function. [*Example:*

```
void (*p)() throw(int);
void (**pp)() noexcept = &p; // error: cannot convert to pointer to noexcept function

struct S { typedef void (*p)(); operator p(); };
```

```
void (*q)() noexcept = S(); // error: cannot convert to pointer to noexcept function
```

— *end example*]

5.3.7 noexcept operator [expr.unary.noexcept]

1 The `noexcept` operator determines whether the evaluation of its operand, which is an unevaluated operand (Clause 5), can throw an exception (15.1).

noexcept-expression:

`noexcept` (*expression*)

2 The result of the `noexcept` operator is a constant of type `bool` and is a prvalue.

3 The result of the `noexcept` operator is `true` unless the *expression* is potentially-throwing (15.4) ~~if the set of potential exceptions of the expression (15.4) is empty, and `false` otherwise~~.

8.3.5 Functions [dcl.fct]

2 In a declaration `T D` where `D` has the form

`D1 (` *parameter-declaration-clause* `)` *cv-qualifier-seq*$_{opt}$
ref-qualifier$_{opt}$ *exception-specification*$_{opt}$ *attribute-specifier-seq*$_{opt}$ *trailing-return-type*

and the type of the contained *declarator-id* in the declaration `T D1` is "*derived-declarator-type-list* `T`", `T` shall be the single *type-specifier* `auto`. The type of the *declarator-id* in `D` is "*derived-declarator-type-list* `noexcept`$_{opt}$ function of (*parameter-declaration-clause*) *cv-qualifier-seq*$_{opt}$ *ref-qualifier*$_{opt}$ returning `U`", where `U` is the type specified by the *trailing-return-type*, and where the optional `noexcept` is present if and only if the ~~*exception-specification*~~exception specification (15.4) is non-throwing. The optional *attribute-specifier-seq* appertains to the function type.

8 The return type, the *parameter-type-list*, the ref-qualifier, the *cv-qualifier-seq*, and whether the function has a non-throwing ~~*exception-specification*~~exception specification, but not the default arguments (8.3.6) ~~or the exception specification (15.4)~~, are part of the function type. [*Note:* Function types are checked during the assignments and initializations of pointers to functions, references to functions, and pointers to member functions. — *end note*]

8.4.2 Explicitly-defaulted functions [dcl.fct.def.default]

3 If a function that is explicitly defaulted is declared with an *exception-specification* that is not the same as~~compatible (15.4) with~~ the exception specification of the implicit declaration, then

(3.1) — if the function is explicitly defaulted on its first declaration, it is defined as deleted;

(3.2) — otherwise, the program is ill-formed.

4 [*Example:*

```
struct S {
  constexpr S() = default;            // ill-formed: implicit S() is not constexpr
  S(int a = 0) = default;             // ill-formed: default argument
  void operator=(const S&) = default; // ill-formed: non-matching return type
  ~S() noexcept(false)~~throw(int)~~ = default;          // deleted: exception specification does not match
private:
  int i;
  S(S&);                              // OK: private copy constructor
};
S::S(S&) = default;                   // OK: defines copy constructor
```

— *end example*]

14.5.3 Variadic templates [temp.variadic]

~~(4.7) — In a *dynamic-exception-specification* (15.4); the pattern is a *type-id*.~~

15.3 Handling an exception [except.handle]

7 A handler is considered active when initialization is complete for the parameter (if any) of the catch clause. [*Note:* The stack will have been unwound at that point. — *end note*] Also, an implicit handler is considered active when

`std::terminate()` ~~or `std::unexpected()`~~ is entered due to a throw. A handler is no longer considered active when the catch clause exits ~~or when `std::unexpected()` exits after being entered due to a throw~~.

15.5 Special functions [except.special]

1 The function~~s~~ `std::terminate()` (15.5.1) ~~and `std::unexpected()` (15.5.2) are~~is used by the exception handling mechanism for coping with errors related to the exception handling mechanism itself. The function `std::current_exception()` (18.8.6) and the class `std::nested_exception` (18.8.7) can be used by a program to capture the currently handled exception.

15.5.1 The std::terminate() function [except.terminate]

1 In some situations exception handling must be abandoned for less subtle error handling techniques. [*Note:* These situations are:

(1.1) — when the exception handling mechanism, after completing the initialization of the exception object but before activation of a handler for the exception (15.1), calls a function that exits via an exception, or

(1.2) — when the exception handling mechanism cannot find a handler for a thrown exception (15.3), or

(1.3) — when the search for a handler (15.3) encounters the outermost block of a function with a non-throwing exception specification ~~*noexcept-specification* that does not allow the exception~~ (15.4), or

(1.4) — when the destruction of an object during stack unwinding (15.2) terminates by throwing an exception, or

(1.5) — when initialization of a non-local variable with static or thread storage duration (3.6.3) exits via an exception, or

(1.6) — when destruction of an object with static or thread storage duration exits via an exception (3.6.4), or

(1.7) — when execution of a function registered with `std::atexit` or `std::at_quick_exit` exits via an exception (18.5), or

(1.8) — when a *throw-expression* (5.17) with no operand attempts to rethrow an exception and no exception is being handled (15.1), or

~~(1.9) — when `std::unexpected` exits via an exception of a type that is not allowed by the previously violated exception specification, and `std::bad_exception` is not included in that exception specification (15.5.2), or~~

~~(1.10) — when the implementation's default unexpected exception handler is called (D.5.1), or~~

(1.11) — when the function `std::nested_exception::rethrow_nested` is called for an object that has captured no exception (18.8.7), or

(1.12) — when execution of the initial function of a thread exits via an exception (30.3.1.2), or

(1.13) — when the destructor or the copy assignment operator is invoked on an object of type `std::thread` that refers to a joinable thread (30.3.1.3, 30.3.1.4), or

(1.14) — when a call to a `wait()`, `wait_until()`, or `wait_for()` function on a condition variable (30.5.1, 30.5.2) fails to meet a postcondition.

— *end note*]

2 In such cases, `std::terminate()` is called (18.8.4). In the situation where no matching handler is found, it is implementation-defined whether or not the stack is unwound before `std::terminate()` is called. In the situation where the search for a handler (15.3) encounters the outermost block of a function with a non-throwing exception specification ~~*noexcept-specification* that does not allow the exception~~ (15.4), it is implementation-defined whether the stack is unwound, unwound partially, or not unwound at all before `std::terminate()` is called. In all other situations, the stack shall not be unwound before `std::terminate()` is called. An implementation is not permitted to finish stack unwinding prematurely based on a determination that the unwind process will eventually cause a call to `std::terminate()`.

~~**15.5.2 The `std::unexpected()` function [except.unexpected]**~~

~~1 If a function with a *dynamic-exception-specification* exits via an exception of a type that is not allowed by its exception specification, the function `std::unexpected()` is called (D.5) immediately after completing the stack unwinding for the former function.~~

~~2 [*Note:* By default, `std::unexpected()` calls `std::terminate()`, but a program can install its own handler function (D.5.2). In either case, the constraints in the following paragraph apply. — *end note*]~~

~~3 The `std::unexpected()` function shall not return, but it can throw (or rethrow) an exception. If it throws a new exception which is allowed by the exception specification which previously was violated, then the search for another handler will continue at the call of the function whose exception specification was violated. If it exits via an exception of a type that the *dynamic-exception-specification* does not allow, then the following happens: If the *dynamic-exception-specification* does not include the class `std::bad_exception` (18.8.3) then the function `std::terminate()` is called, otherwise the thrown exception is replaced by an implementation-defined object of type `std::bad_exception` and the search for another handler will continue at the call of the function whose *dynamic-exception-specification* was violated.~~

~~4 [*Note:* Thus, a *dynamic-exception-specification* guarantees that a function exits only via an exception of one of the listed types. If the *dynamic-exception-specification* includes the type `std::bad_exception` then any exception type not on the list may be replaced by `std::bad_exception` within the function `std::unexpected()`. — *end note*]~~

17.6.4.3.1 Zombie names [zombie.names]:

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_unexpected`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref`, `mem_fun_ref_t`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `set_unexpected`, ~~and~~ `unary_function`~~,~~ `unexpected`, and `unexpected_handler`.

17.6.4.7 Handler functions [handler.functions]

1 The C++ standard library provides default versions of the following handler function~~s~~ (Clause 18):

~~(1.1) — `unexpected_handler`~~

(1.2) — `terminate_handler`

2 A C++ program may install different handler functions during execution, by supplying a pointer to a function defined in the program or the library as an argument to (respectively):

(2.1) — `set_new_handler`

~~(2.2) — `set_unexpected`~~

(2.3) — `set_terminate`

See also: subclauses 18.6.3, Storage allocation errors, and 18.8, Exception handling.

3 A C++ program can get a pointer to the current handler function by calling the following functions:

(3.1) — `get_new_handler`

~~(3.2) — `get_unexpected`~~

(3.3) — `get_terminate`

4 Calling the `set_*` and `get_*` functions shall not incur a data race. A call to any of the `set_*` functions shall synchronize with subsequent calls to the same `set_*` function and to the corresponding `get_*` function.

17.6.4.8 Other functions [res.on.functions]

(2.2) — for handler functions (18.6.3.3, 18.8.4.1~~, D.5.1~~), if the installed handler function does not implement the semantics of the applicable *Required behavior:* paragraph.

17.6.5.12 Restrictions on exception handling [res.on.exception.handling]

1 Any of the functions defined in the C++ standard library can report a failure by throwing an exception of a type described in its **Throws:** paragraph. ~~An implementation may strengthen the exception specification for a non-virtual function by adding a non-throwing *noexcept-specification*.~~

2 A function may throw an object of a type not listed in its **Throws** clause if its type is derived from a type named in the **Throws** clause and would be caught by an exception handler for the base type.

3 Functions from the C standard library shall not throw exceptions[188] except when such a function calls a program-supplied function that throws an exception.[189]

4 Destructor operations defined in the C++ standard library shall not throw exceptions. Every destructor in the C++ standard library shall behave as if it had a non-throwing exception specification. Any other functions defined in the C++ standard library that do not have an *exception-specification* may throw implementation-defined exceptions unless otherwise specified.[190] ~~An implementation may strengthen this implicit *exception-specification* by adding an explicit one~~.[191]

5 An implementation may strengthen the exception specification for a non-virtual function by adding a non-throwing exception specification.

~~191) That is, an implementation may provide an explicit *exception-specification* that defines the subset of "any" exceptions thrown by that function. This implies that the implementation may list implementation-defined types in such an *exception-specification*.~~

18.8 Exception Handling [support.exception]

1 The header `<exception>` defines several types and functions related to the handling of exceptions in a C++ program.

Header <exception> synopsis

```
namespace std {
  class exception;
  class bad_exception;
  class nested_exception;

  typedef void (*unexpected_handler)();
  unexpected_handler get_unexpected() noexcept;
  unexpected_handler set_unexpected(unexpected_handler f) noexcept;
  [[noreturn]] void unexpected();

  typedef void (*terminate_handler)();
  terminate_handler get_terminate() noexcept;
  terminate_handler set_terminate(terminate_handler f) noexcept;
  [[noreturn]] void terminate() noexcept;

  int uncaught_exceptions() noexcept;
  // D.9X, uncaught_exception (deprecated)
  bool uncaught_exception() noexcept;

  typedef unspecified exception_ptr;

  exception_ptr current_exception() noexcept;
  [[noreturn]] void rethrow_exception(exception_ptr p);
  template <class E> exception_ptr make_exception_ptr(E e) noexcept;
  template <class T> [[noreturn]] void throw_with_nested(T&& t);
  template <class E> void rethrow_if_nested(const E& e);
}
```

18.8.3 Class `bad_exception` [bad.exception]

1 The class `bad_exception` defines the type of the object~~s~~ referenced by the `exception_ptr` returned from a call to `current_exception` (18.8.6 [propagation]) when the currently active exception object fails to copy~~thrown as described in (15.5.2)~~.

23.3.7.8 Zero sized arrays [array.zero]

4 Member function `swap()` shall have a non-throwing exception specification~~*noexcept-specification* which is equivalent to `noexcept(true)`~~.

Annex B (informative) Implementation quantities [implimits]

~~(2.40) — Throw specifications on a single function declaration [256].~~

C.2.6 Clause 12: special member functions [diff.cpp03.special]

12.4 (destructors)

Change: User-declared destructors have an implicit exception specification.

Rationale: Clarification of destructor requirements.

Effect on original feature: Valid C++ 2003 code may execute differently in this International Standard. In particular, destructors that throw exceptions will call `std::terminate()` (without calling `std::unexpected()`) if their exception specification is non-throwing~~is `noexcept` or `noexcept(true)`. For a throwing virtual destructor of a derived class, `std::terminate()` can be avoided only if the base class virtual destructor has an exception specification that is not `noexcept` and not `noexcept(true)`~~.

C.4.x Clause 15: Exception handling [diff.cpp14.except]

15.4

Change: Remove dynamic exception specifications.

Rationale: Dynamic exception specifications were a deprecated feature that was complex and brittle in use. They interacted badly with the type system, which is a more significant issue in this International Standard where exception specifications become part of the function type.

Effect on original feature: A valid C++ 2014 function declaration, member-function-declaration, function-pointer-declaration, or function-reference declaration, which has a potentially throwing dynamic exception specification will be rejected as ill-formed in this International Standard. Violating a non-throwing dynamic exception specification will call `terminate` rather than `unexpected` and might not perform stack unwinding prior to such a call.

D.2 Dynamic exception specifications [depr.except.spec]

1 The *exception-specification* `throw()`~~use of *dynamic-exception-specifications*~~ is deprecated.

~~**D.5 Violating *exception-specifications* [exception.unexpected]**~~

~~**D.5.1 Type `unexpected_handler` [unexpected.handler]**~~

```
typedef void (*unexpected_handler)();
```

~~1 The type of a *handler function* to be called by `unexpected()` when a function attempts to throw an exception not listed in its *dynamic-exception-specification*.~~

~~2 *Required behavior:* An `unexpected_handler` shall not return. See also 15.5.2.~~

~~3 *Default behavior:* The implementation's default `unexpected_handler` calls `std::terminate()`.~~

~~**D.5.2 `set_unexpected` [set.unexpected]**~~

```
unexpected_handler set_unexpected(unexpected_handler f) noexcept;
```

~~1 *Effects:* Establishes the function designated by `f` as the current `unexpected_handler`.~~

~~2 *Remark:* It is unspecified whether a null pointer value designates the default `unexpected_handler`.~~

~~3 *Returns:* The previous `unexpected_handler`.~~

~~**D.5.3 `get_unexpected` [get.unexpected]**~~

```
unexpected_handler get_unexpected() noexcept;
```

~~1 *Returns:* The current `unexpected_handler`. [*Note:* This may be a null pointer value. — *end note*]~~

~~**D.5.4 `unexpected` [unexpected]**~~

```
[[noreturn]] void unexpected();
```

~~1 *Remarks:* Called by the implementation when a function exits via an exception not allowed by its *exception-specification* (15.5.2), in effect after evaluating the *throw-expression* (D.5.1). May also be called directly by the program.~~

~~2 *Effects:* Calls the current `unexpected_handler` function. [*Note:* A default `unexpected_handler` is always considered a callable handler in this context. — *end note*]~~

Acknowledgements

Thanks to Jens Maurer for the initial review, identifying a number of proposed edits that had unintended consequences.

VIII. References

- N0741 Exceptions specifications in the Standard Library, Mats Henricson
- N3051 Deprecating Exception Specifications, Doug Gregor
- N4086 Removing trigraphs??!, Richard Smith
- N4168 Removing `auto_ptr`, Billy Baker
- N4190 Removing `auto_ptr`, `random_shuffle()`, And Old `<functional>` Stuff, Stephan T. Lavavej
- P0001R1 Remove Deprecated Use of the `register` Keyword
- P0002R1 Remove Deprecated `operator++(bool)`
- P0004R1 Remove Deprecated iostreams aliases
- P0012R1 Make exception specifications be part of the type system, version 3, Jens Maurer